01
File System Fundamentals

Concepts, visuals, and a tested C11 file-I/O demonstration. This lesson introduces the vocabulary used by the later disk, superblock, inode, directory, and CLI modules.

MODULE	FOCUS
Prerequisites	Basic C syntax, functions, and command-line usage
What you will build	A standalone program that creates, reads, appends to, and removes a text file
What this is not	A custom file system implementation - the code uses the host operating system

Learning goal

Understand the difference between a file system concept, an API call that uses an existing file system, and an implementation that manages blocks and metadata itself.

Suggested study time: 45-75 minutes.

Learning objectives and study map

After this lesson, you should be able to:

• Define a file system and identify why an operating system needs one.

• Explain files, directories, paths, names, metadata, and permissions.

• Distinguish a logical file hierarchy from physical storage placement.

• Describe how a C program interacts with an existing file system.

• Compile and test safe create, read, append, and delete operations.

Reading path

Step	Topic	Result
1	File systems and terminology	Understand the abstraction
2	Directories and paths	Interpret a file tree
3	Storage overview	Recognize blocks and metadata
4	C demo	Observe file operations
5	Questions and next steps	Check understanding

Key idea

The file system is responsible for naming files, arranging directories, tracking information about files, and mapping logical operations to storage. A program typically asks the operating system to perform file operations; it does not access disk hardware directly.

1. What is a file system?

A file system is a set of structures and rules used to organize, store, find, update, and delete data on a storage medium. It provides names and directories so programs can work with files without needing to know exact physical storage locations.

Why it is needed

• Organization: represent documents in folders and subfolders.

• Lookup: resolve a path or name to the intended file.

• Storage management: track used and free capacity.

• Metadata: record file size, type, timestamps, and other properties.

• Access control and reliability: apply available permission rules and manage consistency.

Useful terms

Term	Meaning
File	Named collection of data, often treated as a byte stream.
Directory	Organizes names and references to files or other directories.
Path	Text indicating a location within a directory hierarchy.
Metadata	Information about a file, such as size and timestamps.
Block	Fixed-size unit used in many storage designs.
File system API	Operations offered to programs to access an existing file system.

Examples of real file systems include FAT32, NTFS, and ext4. Their internal implementations differ; this lesson focuses on common high-level ideas.

2. Directory tree and paths

A hierarchical file system arranges entries beneath a root directory. A directory can contain ordinary files or further directories.

```
/
  |-- documents/
  |     |-- notes.txt
  |     `-- report.pdf
  `-- images/
        `-- photo.png
```

Absolute vs. relative paths

Path	Interpretation
/documents/notes.txt	An absolute POSIX-style path starting at root.
documents/notes.txt	A relative path, resolved against a current directory.
../images/photo.png	Move to parent, then down into images (when valid).

Path syntax differs across operating systems. A Windows path may use a drive letter and backslashes, for example `C:\Users\...`.

File name is not file content

The name `notes.txt` identifies an entry in a directory; it is not the same as the file's bytes. Similarly, a directory organizes names but does not necessarily store all children's content inside one contiguous physical region.

3. Logical view vs. storage layout

From an application's perspective, a file is a named stream of bytes. Internally, a file system may divide storage into fixed-size blocks and keep metadata to find those blocks. The mapping depends on the file system design.

Illustrative logical block map: 512 bytes per block; block indices start at zero.

Illustration - converting block numbers to byte offsets

If the storage uses 512-byte blocks and numbering starts at zero, the start of logical block 3 is:

```
offset = block_number * block_size
offset = 3 * 512
offset = 1536 bytes
```

This example is a calculation, not a claim about how any particular real file system lays out its data. Your simulator will later define its own block size and allocation mechanism.

What is metadata?

Example metadata	Purpose
File size	How many bytes of content are currently present
Timestamp	When the file was modified (if supported)
Block mapping	Where file content is stored in the chosen layout
Permissions	Which operations are allowed (if supported)

A later lesson will introduce the superblock, free-space bitmap, and inode-like metadata in the custom simulator.

4. What the C demonstration does

The included program `fs_intro_demo.c` is a working example of using the host operating system's file system through the C standard library.

Sequence

Step	Call / mode	Purpose
1	fopen(path, "wx")	Create only if the demo file does not already exist; C11 exclusive creation.
2	fputs(...)	Write an initial line.
3	fgets(...)	Read and print file lines until end of file.
4	fopen(path, "a")	Append a second line without replacing earlier contents.
5	fclose(...)	Close streams and check errors.
6	remove(path)	Delete the generated file when --clean is requested.

Selected core logic

```
if (add_text(DEMO_FILE, "wx", "Hello from a C file!\n") != 0)
    return 1;

if (show_contents(DEMO_FILE) != 0) return 1;

if (add_text(DEMO_FILE, "a", "This line was appended.\n") != 0)
    return 1;
```

The complete code is in the accompanying source file. The exclusive `"wx"` mode helps prevent accidental overwrites. The program checks opening, reading, writing, and closing operations for errors.

Important limitation

The demo does not implement sectors, blocks, inodes, directory entries, journaling, or a virtual disk. Those features belong to the later project. It uses existing OS facilities instead.

5. Build, run, inspect, clean

Compile (Linux, macOS, or GCC on Windows)

```
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 \
    fs_intro_demo.c -o fs_intro_demo
```

Run from the lesson folder

```
./fs_intro_demo
```

Expected output

```
1. Created a file and wrote the first line.
Contents of fs_intro_sample.txt:
Hello from a C file!
2. Appended another line.
Contents of fs_intro_sample.txt:
Hello from a C file!
This line was appended.
3. The file remains on disk as fs_intro_sample.txt.
4. Run the program with --clean to remove the demo file.
```

Clean up

```
./fs_intro_demo --clean
```

On Windows, the executable typically runs as `fs_intro_demo.exe`. If you run the program twice without cleaning, the second run should refuse to overwrite the existing sample file. This is intentional.

Test checklist

- First execution creates the sample file and displays both lines.
- The sample file contains exactly the displayed two lines.
- Second execution refuses to overwrite the sample file.
- Running with `--clean` removes the sample file.

6. Practice, review, and next lesson

Mini exercises

• Draw a tree containing `/courses/os/notes.txt` and `/courses/c/hello.c`.

• Calculate the starting offset of block 7 if block size is 1024 bytes.

• Explain what changes between opening a file in `"a"` and `"w"` mode.

• Run the example twice without cleaning. Explain why the second run fails.

• Identify which lines in the demo create, write, read, append, and delete.

Short answer key

2: 7 * 1024 = 7168 bytes. 3: Append mode writes at the end of the file; write mode ordinarily truncates an existing file. 4: The C11 exclusive-create mode refuses to create an already existing file.

Common misconceptions

Misconception	Correction
"fopen makes my own file system"	fopen uses the host OS and its file system.
"A path is the physical block address"	A path is a logical name resolved by the file system.
"Deleting always erases every underlying byte"	Deletion semantics vary; freeing a file does not guarantee secure erasure.

Next: Learning 02 - Disk and Block Management

You will move from using an existing file system toward simulating a virtual disk in a regular file, defining a block size, calculating offsets, and performing controlled block-level reads and writes.

Further reading

C11 file I/O: C standard library documentation for `fopen`, `fgets`, `fputs`, `fclose`, and `remove`. For deeper systems context: Operating Systems: Three Easy Pieces (file systems chapters).